A RESEARCHED PATH FROM LEARNING TO EVIDENCE

Build evidence. *Earn trust.*

An AI & ML engineering portfolio plan, from your first program to work an employer can inspect.

- 7 November 2026 / Open portfolio field guide

DESIGNED FOR A BEGINNER'S STARTING POINT

Experience	**Starting from zero**
Study budget	**6 hours a day**
Job search	**Global, eligibility first**

Build something useful. Measure it honestly. Make your work easy to inspect.

02	**Flagship projects** Complementary proof of ability	01	**Reliability release** Evidence that the system holds up	20	**Postings reviewed** A targeted hiring sample

A PLAN YOU CAN USE

Build your portfolio plan.

Choose a direction, set a sustainable study budget, and track the evidence you can demonstrate. Each step links to its full project brief.

Start with the foundation. Build the two complementary flagships in an order that fits your eligible target roles, improve one system's reliability, and prepare a contribution and case study.

The guide's default allowance is 24 weeks at 36 study hours a week. Adjust the calendar to your circumstances and move on when you can demonstrate the gate.

01 **Own a small Python/data tool**

Gate: add a CSV validation rule and a regression test, debug a malformed input, and explain the code independently. Build the software/data prerequisites before either flagship.

02 **Demand Replay · forecasting**

Gate: reproduce a baseline comparison, defend the prediction-time boundary, and explain leakage checks and observed errors. Requires the software/data foundation and basic regression/evaluation.

03 **Evidence Desk · grounded answers**

Gate: reproduce retrieval and answerability evaluations, show citations and abstention, and explain a remaining failure. Requires API, text-processing and evaluation fundamentals.

04 **Make one flagship recoverable**

Gate: demonstrate bounded failures, useful logs and a rollback drill on a working flagship. Report measured local conditions and monitoring limits.

05 **Contribute, explain and apply**

Gate: show a useful contribution with its actual status, a reproducible case study, and evidence matched to eligible roles. Practice an unfamiliar code change independently.

Research and recommendations prepared **7 November 2026** for learners starting from zero, with a planning budget of six study hours per day and a global job-search goal. Adapt the plan to your actual education, work authorization, hardware and budget. The default assumes a normal laptop, no GPU, and a free-first build. Estimates are planning budgets after prerequisite skills, not promises of learning speed or employment.

Research provenance: This guide was prepared with AI assistance, source review and a direct dataset-schema inspection. It proposes projects; it does not claim the publisher has built these systems, measured their performance or obtained employment through them. The dated source records and limitations are included for review.

1. The recommendation

Build **two complementary flagship projects**, give one a **serious reliability release**, and make **one useful contribution to an existing project**. Start with a small Python/data tool so you can actually own the work. Publish the evidence in a form that a hiring team can inspect and that you can explain without depending on generated answers.

ORDER	WHAT TO BUILD	WHAT AN EMPLOYER CAN INSPECT	WHERE IT BELONGS
Foundation	A CSV quality-checking CLI	Your Python, error handling, tests, Git habits, and ability to finish a small tool	A learning repository; pin only while it is your strongest work
Flagship 1	**Demand Replay:** a leakage-aware next-day demand forecast and API	Data contracts, prediction-time reasoning, simple baselines, temporal evaluation, and reproducible model delivery	A pinned GitHub repository, case study, and short demo
Flagship 2	**Evidence Desk:** a bounded documentation assistant with citations and abstention	Retrieval, LLM integration, evaluation design, grounded answers, failure analysis, cost and latency	A pinned GitHub repository, evaluation report, and short demo
Reliability release	A monitored, bounded, recoverable version of one flagship	Timeouts, schema failures, observability, load testing, release controls, rollback, and an incident report	A tagged release and case study inside the flagship repository
Collaboration	A verified correction, regression test, or small bug fix in a tool you actually use	Reproduction, review response, communication, and working within someone else's codebase	Link to the real issue/PR and a precise contribution summary

Default sequencing: foundation → Demand Replay → Evidence Desk → reliability release. This puts data and evaluation discipline before LLM orchestration. If your eligible target roles clearly emphasize AI product engineering, you can build Evidence Desk before the forecasting flagship once the software/evaluation prerequisites are met. The choice follows the target work, not a trend.

Do not create a separate repository for every component. Two coherent, reproducible systems with documented decisions provide more inspectable evidence than a large collection of unfinished interfaces. The separate First Principles learning roadmap is a related resource. A roadmap or documentation application can show interface design, packaging and delivery; it does not by itself establish independently trained models, valid ML experiments or professional production experience.

2. What the research actually supports

Method and limits

The hiring review examined **20 distinct postings from 11 employers**: 17 directly about AI/ML and three adjacent entry-level software roles. The employer-controlled feeds were captured on 6 November 2026 and reviewed on 7 November. The employers are Databricks, Datadog, Elastic, Anthropic, Reddit, Pinterest, Roblox, Duolingo, Airbnb, Scale AI, and Affirm. A separate appendix preserves titles, locations, exact evidence, requirements, warnings, and URLs.

This is a **purposive capability sample**, heavily weighted toward large technology companies and accessible English-language job boards. It is not a representative global labor-market survey. It cannot estimate the share of employers requiring a degree, your chances of getting hired, the availability of worldwide remote jobs, or the best project for every country. A captured job description is also not a guarantee that the vacancy remains open. One posting contains unfinished template text; ambiguous conditions from that text must not be used as eligibility evidence.

The review also uses first-party technical guidance, employer interview guidance, official dataset/documentation pages, GitHub/LinkedIn documentation, and broader labor-market reports. Evidence is linked below. The project selection, time budgets, readiness gates, and publishing cadence are recommendations in this guide, not an experimentally validated hiring formula.

The clearest hiring signals

OBSERVED EVIDENCE	WHAT IT MEANS FOR YOUR PORTFOLIO	BOUNDARY
Databricks describes production GenAI deployment, evaluation, and optimization; its “all levels” FDE posting also explicitly asks for at least four years of relevant experience. H01	Show a complete, evaluated workflow and customer-facing explanation.	“All levels” does not mean suitable for a beginner.
Databricks welcomes infrastructure engineers without an ML background while emphasizing software testing and deployment. H02	Strong software fundamentals are a useful bridge into applied AI.	“No ML background” is not “no engineering experience.”
Anthropic's demo-engineering role says accuracy and honesty matter alongside polish and mentions evaluation/validation harnesses and graceful fallbacks. H05	A beautiful demo should expose reliable behavior, limits, and failure handling.	This is an experienced engineering role, not a promise that a demo qualifies you.
Reddit describes ownership from problem definition and feature engineering through training, deployment, and monitoring. H06	Build the chain around the model, with a reproducible baseline and a service/replay.	The posting also requests production experience.

OBSERVED EVIDENCE	WHAT IT MEANS FOR YOUR PORTFOLIO	BOUNDARY
Scale AI asks for extending evaluations, investigating failure modes, flagging regressions, and owning work through release/debugging. H13	Publish experiments, failures, regression checks, and the release decision.	A portfolio replay should not be described as professional production ownership.
Pinterest's master's graduate role explicitly asks for critical validation of AI-generated suggestions. H09	Explain your AI assistance and demonstrate independent debugging and judgment.	This particular role also has an explicit master's requirement.
Affirm's early-career SWE role accepts equivalent practical experience or a related bachelor's degree. H18	Junior software work can be a relevant adjacent route while you develop ML depth.	Its location and authorization conditions still apply.

The practical conclusion is to demonstrate **problem definition, data judgment, evaluation, readable software, delivery, and independent explanation**. Framework familiarity is useful when it supports those behaviors. A framework list alone gives a reviewer little evidence.

Eligibility comes before portfolio matching

Several reviewed student/new-graduate roles are restricted by degree, enrollment, graduation date, or location. Pinterest's master's graduate role, Roblox's PhD early-career role, Duolingo's new-PhD role, and the Datadog internship illustrate different restrictions; they do not establish a universal degree rule. H09 H10 H11 H20

“Remote” is also scoped: the reviewed Pinterest role is US-based, while Affirm's MLE role specifies Canadian eligibility details. Airbnb's China role explicitly requires English and Mandarin. Scale AI's public-sector role requires an active security clearance. These are not gaps you can fix by adding another notebook. H07 H16 H12 H14

If you are not enrolled in a qualifying degree, filter student-only roles out of your primary list. Include eligible junior software/backend, data-platform, QA automation, or applied-AI implementation roles where the work and requirements match your actual skills. This is a search strategy to test in your market, not a claim that those jobs are easy or available everywhere.

Demand and the future

The retrieved US BLS data-scientist page projects 35% employment growth in 2025–2035; its software developer/QA/tester group projects 10%. These are broad US occupational projections, with openings including replacement needs. They are not forecasts of globally accessible junior AI-engineering vacancies or individual job chances. S01 S02

WEF's 2025 report summarizes expectations from more than 1,000 employers across 55 economies and highlights AI/big data alongside analytical thinking and other broader capabilities. Those are surveyed expectations through 2030. Stanford's 2026 economy chapter describes uneven labor effects, including

pressure in young-worker hiring pipelines; it does not prove that a particular applicant will be helped or displaced by AI. S03 S04

Planning principle: as tools change, keep investing in understanding the problem, constructing valid tests, tracing data, measuring constraints, integrating systems, and communicating decisions. If code generation improves, those capabilities help you check and own its output. If budgets tighten, a measured baseline and a smaller dependable solution become valuable. These are conditional planning arguments, not predictions of a winning framework or a hiring boom.

---

3. Choose the job family before choosing extra tools

TARGET	MAIN ENGINEERING QUESTION	PORTFOLIO EMPHASIS	ADDITIONAL PREPARATION
Applied AI / AI product engineer	Can you turn a model capability into a bounded, useful workflow?	Evidence Desk, evaluations, API integration, user feedback, cost/latency, failure handling	Python or TypeScript, HTTP, SQL, basic frontend, auth/secrets, retrieval and LLM failure modes
ML engineer	Can you make predictions that remain valid and useful after training?	Demand Replay, prediction-time data boundaries, temporal evaluation, deployment, model/data monitoring	Python, SQL, probability/statistics, feature pipelines, optimization basics, software testing
ML platform / MLOps-oriented engineer	Can other people train, deploy, diagnose, and recover a system reproducibly?	Reliability release, artifact versioning, CI, containers, observability, rollback	Stronger Linux/networking/cloud/automation and distributed-system fundamentals
Research engineer / research scientist	Can you reproduce and extend a technical result rigorously?	A later replication with ablations and uncertainty, plus mathematical/experimental depth	Often a substantially higher research/degree bar in this sample; not the default first target from zero

Titles overlap. Read responsibilities and requirements rather than treating this table as a universal job taxonomy. For your starting point, I would build a common software/data base, then orient the first case study toward **eligible applied-AI or junior software roles**, while the forecasting project supplies genuine ML evidence. Keep the ML-engineering path open rather than claiming research readiness prematurely.

A repeatable project-selection rule

Use this process again when your target or resources change:

1. **Define the opportunity:** collect a small recent set of roles you could actually accept. Separate location, authorization, degree/enrollment and experience constraints from preferred tools. Record unknowns instead of assuming eligibility.
2. **Extract observable work:** turn responsibilities into things a reviewer could inspect—for example, a defensible train/test boundary, a failing-input regression test, a measured retrieval comparison, or a recovery procedure.
3. **Reject infeasible ideas:** require lawful usable data, a baseline, a measurable outcome, and a build you can run within your available hardware and money. An exciting idea without these is a research question, not yet a portfolio commitment.
4. **Choose complementary evidence:** pick one project with the strongest match to repeated responsibilities, then a second that proves an important missing capability. Prefer depth on an existing system over a third project that repeats the same API wrapper.
5. **Release the smallest useful version:** run the baseline and one real evaluation before expanding the interface or model stack. Stop adding features when you cannot explain or validate the current ones.
6. **Update from observations:** after a release, use actual errors, user feedback, technical reviews and application-stage feedback to choose the next change. Recheck role requirements periodically because job pages and eligibility rules change.

This is a decision procedure, not a mathematically proven optimum. I chose forecasting because it makes data timing and ML evaluation visible, and a documentation assistant because it makes retrieval, generative-model integration and evidence handling visible. Their reliability work shares software skills while exposing different model failures. A third chatbot would add relatively little evidence at this stage.

---

4. Foundation project: a CSV quality-checking CLI

Purpose: prove that you can save, run, test, document, and repair a useful Python program before composing larger AI systems. A small data team wants to know whether a daily file is safe to ingest.

Build: a command-line tool that reads a small CSV, validates an explicit schema, reports missing/duplicate/invalid values, and writes a machine-readable summary. Use a tiny synthetic fixture first, then the bike dataset described below. Keep the original file unchanged.

MVP contract: required columns are declared; dates parse; counts are nonnegative integers; duplicate dates are reported; malformed rows produce an actionable message and a nonzero exit code; a valid input produces deterministic JSON. Add one test for each failure, plus a clean-input test. Document exactly which checks are and are not implemented.

Learn through it: functions, loops, dictionaries, exceptions, files, paths, command-line arguments, tests, virtual environments, Git, and a readable README. Add SQLite only after the file tool is understandable; a database is not needed to prove the first contract.

Publish when: another person can run the documented command in a fresh environment and you can explain the parsing, validation, and error paths from the code. A reasonable initial work allowance is 30–60 focused hours after basic Python exercises; extend it if the independent explanation gate is not met.

Independent gate: close the assistant/tutorial, add one new validation rule and a regression test, and explain why it belongs at that boundary. Do not replace this with collecting a course certificate.

---

5. Flagship 1: Demand Replay

Problem, audience, and honest scope

A hypothetical bike-service planner wants a next-day usage estimate to prepare staffing or capacity. You will build a **historical demand-planning replay**, not claim a live city deployment or current demand prediction. The dataset is the UCI Bike Sharing collection from 2011–2012. Its official page identifies CC BY 4.0; preserve attribution and the source/version record. S13

During this guide's AI-assisted preparation, the actual official ZIP was inspected to check feasibility: `day.csv` has **731 rows**, `hour.csv` has **17,379 rows**, and the daily dates span 1 January 2011 through 31 December 2012. No duplicate dates or empty daily cells were found in that specific file. In every daily row, `cnt = casual + registered`. This was a schema/feasibility inspection, not a trained-model experiment or a performance result. The archive hash is included in the evidence files.

The dataset is familiar; your differentiator is the forecast contract, leakage control, reproducibility, and delivery. Its age, small size, and single-system context are material limitations. A later extension can use a richer openly licensed dataset after you can defend the small system.

Fix the prediction contract before modeling

At the end of day t, predict total rentals for day t+1 using only information available by the end of t. The actual previous day's count may enter the next day's forecast in the historical replay. Model parameters remain frozen during the final test unless a separately specified retraining protocol is being evaluated.

Assume each daily count becomes available at that day's close for this experiment. UCI does not supply arrival timestamps, so this availability is an explicit replay assumption, not a verified operational service guarantee. A real deployment would need its actual reporting delay built into the feature contract.

Allowed MVP inputs: calendar information known in advance, counts lagged from the forecast origin, and trailing summaries computed only from past observations. Exclude the target day's actual weather, temperature, humidity, and wind unless you have genuine historical weather forecasts available at prediction time. A model using observed future weather must be labeled as a different, oracle-information experiment.

Never use same-day `casual` or `registered` to predict `cnt`: those fields contain components of the answer. Avoid arbitrary row IDs as predictive features. A pipeline can run perfectly while answering the wrong prediction-time question. Scikit-learn's guidance on leakage and preprocessing is directly relevant

here. S06

Build and evaluate in this order

1. **Data contract:** validate schema, units, dates, duplicates, missingness, and target composition. Save the source URL, license, download date, and checksum.
2. **Exploratory analysis on the training period:** inspect seasonality, growth, holidays, zeros, and unusual days. Record hypotheses before trying models.
3. **Simple baselines:** last observed day; same weekday one week earlier; a trailing average. Choose formulas that use only available history.
4. **One simple trained model:** regularized linear regression with calendar/lag features; then at most one tree-based candidate such as histogram gradient boosting. Establish why the added complexity might help.
5. **Time-aware validation:** use chronological splits and, when useful, several expanding-window checks inside the training/development period. Fit preprocessors on training data only. A time split alone does not repair an incorrectly shifted feature. S07
6. **Locked final test:** freeze the chosen feature/model configuration and evaluate once on the untouched period. Repeatedly changing the model after seeing test errors turns that period into development data.
7. **Service and replay:** a small FastAPI service with a health endpoint, validated request/response schema, model/data version metadata, and a local historical-replay command. Reuse one feature-building implementation across training, replay and serving to avoid inconsistent inputs. A simple interface is enough.

Concrete initial split, assigned by target date: train through 30 June 2012 (547 daily rows), validate July–September 2012 (92), and test October–December 2012 (92), before any lag warm-up exclusions. Record exclusions explicitly. Check that no training example's target crosses the cutoff even when its feature date precedes it. A rolling one-day-ahead replay can use previously observed test-period history at each origin; it is not a 92-day-ahead forecast from 30 September. Show a timeline in the evaluation report.

Metrics that answer a decision

METRIC	DEFINITION/USE	PITFALL TO AVOID
MAE	Mean of `abs(actual − prediction)` over the declared forecast rows	An overall average can hide holiday or peak-period failures.
WAPE	`sum(abs(actual − prediction)) / sum(actual)` on a stated set	Undefined when the denominator is zero; aggregation can hide small-group errors.
Slice error	The same metric by weekday/holiday/season or a predeclared demand band	Tiny slices are uncertain; never present one good slice as the overall result.

METRIC	DEFINITION/USE	PITFALL TO AVOID
Prediction-interval coverage and width	Optional: fraction of actuals inside the interval, and interval size	Check time/slice calibration; intervals that always contain everything are unhelpful.
Decision-cost replay	Optional: a clearly labeled cost function for under/over-preparation	Assumed staffing/inventory costs do not establish real business savings.

Report baseline and candidate results on the same rows. If you estimate uncertainty, respect temporal dependence—for example, a justified block-based comparison rather than pretending daily errors are independent. Keep the first release understandable; point estimates, row-level predictions, and honest limitations are better than a misapplied significance test.

There is no pre-earned target score. To claim that ML adds value, demonstrate a meaningful improvement against the baseline under the declared protocol and constraints. If it does not, ship or recommend the baseline and explain what you learned. Do not set a public “20% improvement” claim before measuring it.

What to publish

- A real replay demo showing a forecast origin, available inputs, prediction, later observed outcome, and model version.
- `reports/predictions.csv`, an actual metrics JSON, a baseline comparison, and an error-slice report tied to a commit/configuration.
- A data card, split diagram, leakage checklist, and a short explanation of why target-day weather is excluded.
- Unit/data-contract tests, one API integration test, a clean-clone run command, a small sample mode, and CI that actually executes them.
- An architecture page and three observed failure cases. Add a one-page “why I kept/rejected the more complex model” decision note.

A useful work allowance: 100–160 focused hours after prerequisites, with further time for revisions or unfamiliar concepts. CPU-only scikit-learn, pandas or Polars, SQLite if useful, pytest, FastAPI, and one container are sufficient. Start locally; paid cloud infrastructure is optional.

Interview checks: Why was a random split inappropriate? Which feature could accidentally contain the answer? What exactly was known at the forecast origin? Why might a more accurate aggregate model still be a worse operational choice? What would you monitor if labels arrived late? You should be able to answer with your code and recorded examples.

6. Flagship 2: Evidence Desk

A useful bounded assistant

Build an assistant for a narrow set of beginner Python questions about files, CSV/JSON, exceptions, and command-line arguments. Use a versioned subset of official Python documentation. An answer must cite the supporting page/section, or state that the supplied corpus does not support an answer.

The original application code can use MIT; the Python documentation prose is under the PSF license, while specified code examples also have a 0BSD option. Do not relabel all documentation as MIT or public domain. Pin a snapshot/hash, preserve required notices, record transformations, and keep versioned citation URLs. The moving `/3/` URL is not by itself a reproducibility pin. S14 S15

User promise: "Find and explain an answer supported by this pinned documentation subset." Do not promise a universally correct Python tutor. Treat unfamiliar versions, absent pages, ambiguous questions, and unsupported claims as first-class cases. A foundation model may already know public Python documentation; evaluation must check use of the supplied evidence, not just fluent recall.

Architecture and baselines

Start with `documents → normalized sections → lexical retrieval → cited passages`. This is a useful no-paid-model baseline. Then compare dense retrieval, hybrid retrieval, and optional reranking only when an error category motivates them. Add a constrained generation adapter after you can evaluate retrieval.

The first interface can be a small web page and FastAPI backend. Keep the model adapter interchangeable. CI should use deterministic fixtures/fakes for behavior; clearly separate those from live-model quality evaluations. No API key belongs in browser code. Provide a keyless passage-search/sample mode so a reviewer can inspect the project without spending money.

A useful comparison ladder is:

1. Keyword/BM25 retrieval with direct evidence display.
2. Dense retrieval with the same corpus and evaluation set.
3. Hybrid retrieval or a reranker, if justified by measured retrieval failures.
4. A grounded answer generator, evaluated for answer quality and citation support.
5. Optional model/latency/cost comparison under the same frozen protocol.

Do not start with an autonomous multi-agent framework. This task has a known workflow and bounded data. Anthropic's engineering guidance distinguishes workflows from agents and recommends simpler, composable approaches before unnecessary complexity. S09

Design the evaluation before tuning

Create a small, manually reviewed question set with answerable, unanswerable, ambiguous, version-sensitive, and malicious-instruction cases. Record expected supporting sections and an answerability label. A proposed first target is about **120 questions**: 40 development, 20 threshold/calibration, and 60 held-out test questions. Those numbers are a practical workload choice, not a statistically sufficient universal benchmark.

Keep paraphrases and close topic families in the same split. Do not use test questions to choose chunk sizes, prompts, retrieval settings, or abstention thresholds. Add independently authored questions where possible; if volunteers contribute, remove personal information and obtain permission to publish their examples. Synthetic questions are allowed if labeled and checked; they are not evidence of real-user demand.

Include near-miss passages, contradictions between versions, questions with no supporting page, and attempts to make the assistant follow instructions embedded in retrieved text. Treat retrieved content as data. Avoid giving the assistant tools that can execute code, send messages, or make external changes in the first version. OWASP's prompt-injection guidance is useful for designing these boundaries, but a short attack set does not prove security. S10

MEASURE	DEFINE IT THIS WAY	REPORT IT WITH
Retrieval hit@k	Answerable queries for which at least one gold supporting section appears in the top k, divided by answerable queries	k, query count, gold-label method, and whether full evidence requires more than one section
Citation support	Number of assessed factual claims supported by the cited passage divided by assessed factual claims	The claim-segmentation/rating rubric; missing citations counted as unsupported; inter-reviewer disagreements if reviewed twice
Answer correctness	Human-rubric score or pass rate against the expected answer under the pinned docs	Separate answerable/unanswerable slices; do not collapse every behavior into “accuracy”
Unsupported answer rate	Unanswerable queries on which the system nevertheless asserts an answer divided by unanswerable queries	Test set size and cases; zero observed failures in a small set is not zero real-world risk
Answer coverage	Queries for which the system gives an answer divided by all queries; also report answered answerable queries divided by all answerable queries	Correctness/support among answered queries, plus the answerability mixture, so abstaining on everything cannot look successful
Latency and cost	p50/p95 request time and measured tokens/API usage under stated conditions	Hardware, model/provider/date, prompt/output size, concurrency, repetitions, warm/cold state, and price date

An LLM judge can assist triage, but validate its rubric on a human-reviewed subset and publish disagreements. It is not an independent ground truth simply because another model produced the score. OpenAI's evaluation guidance is useful for the process; it does not remove the need to define task-specific

evidence. S08

Ship only what you can demonstrate

Publish an actual evaluation runner, question/label schema, frozen split manifest, row-level outputs, an error taxonomy, and a baseline table. Give three examples: a supported answer, a deliberate abstention, and a remaining failure. Log the configuration/corpus/model versions so another person can reproduce the comparison as closely as the dependencies allow.

Abstention should be calibrated using held-out development/calibration evidence, not a fabricated "confidence" percentage from the generator. A retrieval score is not automatically a probability of correctness. For small test sets, show numerators/denominators and uncertainty rather than theatrical precision.

A useful work allowance: 120–180 focused hours after software/retrieval prerequisites. Start with local lexical search and CPU-friendly embeddings. No GPU is required for that path. If you use a hosted LLM, set a personal hard spending cap, log actual usage, and evaluate a narrow corpus. A suggested optional experiment envelope is US$10–30 total, not a verified provider price quote or a requirement. A local/extractive version can remain free.

Interview checks: How did you choose the split unit? Could the model answer from memorization without retrieving the right version? What does your retrieval score mean? What happens when retrieval finds a plausible but irrelevant paragraph? How did you measure citation support? Why did you choose a fixed workflow? What would convince you to remove the generator?

---

7. The reliability release: make one system inspectably operable

After one flagship is useful, spend a bounded release improving how it behaves when conditions are bad. Keep this work inside that repository so reviewers can trace the same system from baseline to delivery. Do not manufacture another application merely to obtain a third pin.

Google's ML Test Score work treats production readiness as broader than model quality; its Rules of ML emphasize straightforward engineering and useful baselines. Monitoring guidance also distinguishes changes in input distributions from actual prediction-quality degradation. S05 S11 S12

Implement a small operational contract

- **Input boundaries:** schema validation, payload-size limits, explicit versions, missing-value behavior, and useful error messages.
- **Dependency boundaries:** request timeouts, bounded retries where appropriate, rate/spending limits for paid inference, and a documented fallback. A fallback must be distinguishable from a normal model answer.

- **Observability:** structured logs, request IDs, latency/error counters, and model/corpus/configuration IDs. Keep prompts, keys, and personal data out of routine logs unless a justified, consented, redacted design requires otherwise.
- **Release behavior:** a versioned artifact/configuration, a small evaluation gate, and a tested way to restore the last known working version. CI should run affordable deterministic checks; costly quality evaluations can be a separate explicit job.
- **Documentation:** an operator runbook, a system diagram, and an honest incident report from a controlled drill.

Run four controlled drills

DRILL	WHAT TO DEMONSTRATE	WHAT THE REPORT SHOULD SAY
Malformed input or missing data	The service rejects or handles it according to its contract	Exact request/fixture, response, logged event, and regression test
Slow/unavailable dependency	Bounded wait, visible fallback, no endless retry loop	The injected fault, timeout/retry policy, and observed recovery
Bad model/corpus/config release	The evaluation gate or operator detects a problem and restores the prior version	Actual rollback steps, elapsed recovery time in this drill, and remaining manual work
Shifted inputs or delayed labels	Input checks surface a change; quality conclusions wait for appropriate outcomes	What the monitoring can observe, what it cannot establish, and how later labels are joined

Measure p50/p95 latency, errors, and resource use under a declared small load, such as one and five concurrent clients with fixed request sizes. Record actual hardware and sample count. The chosen load is a development experiment, not proof of internet-scale operation. Define your latency/error budget before a run, then report whether you met it.

Do not infer that a model is inaccurate just because a drift detector fired, or accurate because it stayed quiet. Do not automatically retrain from every alert in the first version. Explain the investigation and release policy. If you lack real users and production traffic, call the result a **controlled reliability drill** or **historical replay**, not “production-grade at scale.”

Publishing package: a tagged release, a one-page incident report, an actual before/after measurement table, the failure-injection commands, and a 60–90-second demonstration. Allow roughly 50–90 focused hours after a working flagship; this is an estimate, not a requirement to add a large infrastructure stack.

8. A real contribution and a small feedback loop

Choose an open-source dependency or tool you genuinely used. Reproduce a specific problem or confusing setup step. Read its contribution guide and existing issues. Offer a minimal reproduction, a focused correction, and a test or executed example when appropriate. Respond to the maintainer's review. The official Open Source Guides provide useful process guidance. S25

A meaningful contribution can be a verified setup correction, an example that previously failed, a useful regression test, or a small bug fix. It need not be a new model architecture. Record **submitted**, **reviewed**, and **merged** accurately; an unmerged self-authored PR is not a maintainer endorsement. Avoid mass typo changes or unsolicited rewrites.

Separately, invite a small number of consenting testers to try one realistic task in a flagship. For the documentation assistant, ask them to find an answer and identify its supporting section. Record where they got stuck and one improvement you made. For the forecasting replay, a peer reviewer can inspect whether your prediction-time contract is valid. If no actual operator uses the tool, say so.

A useful first target is three to five testers/reviewers if available. That is a formative usability/review exercise, not a representative user study. Do not claim customer adoption, business savings, or a success rate from a handful of friendly tests. These are proposed activities. Their inclusion does not imply that outreach, user testing, contributions or applications have been completed.

9. Package every claim with its evidence

An employer's practical questions are: can you solve a relevant problem, make sound choices, handle failures, finish usable work, and explain your contribution? Organize each repository so the answer is easy to inspect.

REVIEW PATH	A USEFUL DESIGN TARGET	WHAT SHOULD BE VISIBLE
Quick skim	About 30 seconds	User/problem, one real demo, your contribution, one qualified result or current status, and evaluation/reproduction links
Technical overview	About five minutes	Baseline, split/labels, architecture, three failure cases, measured constraints, and the important tradeoff
Deep review/interview	As needed	Small readable modules, tests, raw outputs, configuration/version records, and an explanation you can modify under questioning

These reading times are presentation heuristics, not empirical claims about recruiter behavior. The point is to reduce the work required to verify you.

A compact evidence bundle

```
README.md
pyproject.toml + a reproducible dependency specification
src/                        actual application/model code
tests/                      behavior, boundaries, data contracts, integration
evals/                      evaluation runner, labels, split manifest
configs/                    explicit experiment/runtime settings
reports/                    measured outputs and row-level predictions
docs/data-card.md           source, license, version, target, exclusions
docs/evaluation.md          baseline, split, metrics, limitations
docs/architecture.md        components and consequential tradeoffs
docs/failure-cases.md        observed errors and attempted repairs
docs/decisions/              short records of choices worth defending
scripts/                    data acquisition, replay, training, evaluation
.github/workflows/          checks that actually execute
LICENSE + THIRD_PARTY_NOTICES.md where needed
```

Keep the structure proportional. An empty folder hierarchy does not improve a small project. A result should identify its code commit, data/corpus checksum, split, configuration, library/model version, seed where applicable, environment, and actual output. For hosted models, record provider/model/date and changing dependencies rather than promising exact deterministic reproduction.

What the README should contain

GitHub's own guidance emphasizes what the project does, why it is useful, how to start, where help is available, and who maintains it. S16 For your projects, add:

1. A one-sentence problem and main scope limit.
2. Links to a real demo/recording, evaluation report, and tested reproduction command.
3. Your exact contribution and any collaborators or AI assistance.
4. A baseline comparison with actual numbers, units, sample size, and split.
5. A small architecture diagram and the main rejected alternative.
6. Three concrete failures, not just “more data would improve it.”
7. Data/model/code rights and dependency requirements.
8. The next experiment that could change your decision.

Give reviewers a small offline/sample mode and saved evaluation output. Label cached, replayed, and precomputed results. Do not make a paid API key, a sleeping cloud endpoint, or a large model download the only way to see evidence.

Example result sentence, with placeholders: “On [N] held-out [queries/days] using [split protocol], [candidate] achieved [actual metric] versus [baseline metric]. This was an [offline/replayed/live] evaluation; [main limitation].” Never publish the placeholders as a claim or fill them with a target you have not measured.

---

10. GitHub and LinkedIn: make the work easy to find

GitHub

A GitHub profile README conventionally lives in a public repository named exactly after the username, with a nonempty root README. GitHub supports up to six pinned repositories/gists combined. You do not need six completed projects to use the feature. S17 S18

Once the evidence exists, use this pin order:

1. Strongest project aligned with your target role: Evidence Desk for applied-AI product roles, or Demand Replay for MLE roles.
2. The complementary flagship.
3. A meaningful collaboration/contribution record or a later distinct project, if genuinely substantial.
4. A substantial documentation or learning application, if you have built one, as evidence of communication, interface design and delivery.

Keep the reliability release inside its flagship. Link the incident/evaluation report prominently. Do not fill pins with scaffolds just to make the profile appear larger. Accurate About descriptions, relevant topics, a working demo link, and genuine release notes improve navigation; none guarantees stars or hiring.

Use a truthful profile sentence. At the beginning: “Learning Python and ML engineering through reproducible projects.” Later, when supported: “Building evaluated AI applications and ML systems with Python; seeking junior/associate engineering opportunities.” A target role should not be presented as current employment.

CI is evidence that the configured checks ran, not that every possible behavior is correct. Publish readable checks and their scope. If a credential is exposed, revoke/rotate it before treating file deletion as a fix; history and copies can retain it. S19 S20

LinkedIn and other social media

LinkedIn's Featured section supports posts, articles, links, and media. Use it to surface the strongest case study, a short demo, and a concise portfolio overview or contribution. The platform documentation supports those capabilities; it does not establish a ranking formula or an optimal posting time. S21

Make a 60–90-second captioned demonstration for each flagship:

- **0–10 seconds:** who has the problem and one realistic input.

- **10–30 seconds:** the actual useful behavior, including evidence/citations or the forecast origin.
- **30–50 seconds:** an unsupported input or injected failure and the bounded response.
- **50–70 seconds:** the baseline/evaluation result, with sample size and context visible.
- **70–90 seconds:** one tradeoff, your contribution, and where to inspect the work.

Use a clean demonstration profile. Label simulations, historical replay, cached answers, and edited timing. Provide a written equivalent. The purpose is to make real evidence understandable, not to hide weaknesses behind a polished video.

Three posts worth publishing when the work exists

An experiment: "I am building [tool] for [specific problem]. The baseline was [method]. On [N, held-out unit], [change] moved [metric] from [actual A] to [actual B], with [cost/latency tradeoff]. The largest remaining error category was [observed issue]. Evaluation and reproduction: [link]."

A failure and repair: "This input broke my [system]. I assumed [assumption]; [trace/test] showed [cause]. I changed [implementation] and added [regression]. It still struggles with [limit]. Before/after evidence: [link]."

A release and feedback: "I released [version] to test [user need]. [Actual number/type] of testers tried [task]; I observed [finding] and changed [behavior]. This is a small formative test, not a production-adoption claim. Demo, evaluation, and limitations: [link]."

If nobody has tried it yet, say that and invite specific feedback. If a model loses to the baseline, explain why the simpler approach wins. Avoid vague "95% accuracy," unmeasured time savings, invented clients, and "production scale" claims from a local test.

Allow about **2–3 hours of a 36-hour work week** for documentation and communication once real artifacts exist. One substantive update a week or every two weeks is a reasonable starting cadence; it is not a quota, an algorithm trick, or a proven hiring intervention. Measure useful reviews, qualified conversations, reproducibility feedback, and application-stage outcomes. Stars and views help distribution, but do not certify skill.

---

11. A 24-week first pass, with evidence gates

This is a planning scaffold for **six focused hours on six days per week**, with one recovery/light-review day. That is 36 scheduled hours weekly. Actual useful time varies; carry unfinished work forward and reduce scope before sacrificing understanding. Completing this calendar is not a job guarantee or a substitute for the gates.

WEEKS	MAIN WORK	PUBLISHABLE OUTPUT	GATE BEFORE MOVING ON
1–2	Python basics, terminal, files, functions, simple tests and Git	Tiny exercises and the first CSV checker version	Add a validation rule and explain its test without copying a solution
3–5	Collections, data cleaning, SQL basics, HTTP/API basics, readable modules	A documented, tested CSV CLI; small SQL/API exercises	Debug malformed input, write a join, and explain a request/response boundary
6–8	Arrays, core algebra/probability/statistics, regression, leakage, evaluation	A small baseline experiment and a split diagram	Explain target, features, baseline, loss/metric, and why the split is valid
9–13	Demand Replay	First flagship repository, actual predictions/metrics, data card and demo	Clean-clone run; honest temporal evaluation; code and failures you can defend
14–18	Evidence Desk	Second flagship, retrieval/answerability evaluation, citation/abstention demo	Held-out queries; reproducible retrieval; documented unsupported answers and costs
19–21	Reliability release and tester feedback	Incident report, regression checks, tagged release, revised case study	Demonstrate failure handling, monitoring limits, and a real rollback drill
22–24	Contribution, interview practice, tailored application loop	Accurate contribution record, profile/Featured assets, targeted applications	Match specific role requirements to evidence and solve a small unfamiliar change independently

The project-hour estimates above are narrower implementation allowances after prerequisites. These calendar blocks also include learning, debugging, documentation, review, and contingency; they are not additional guaranteed durations. Some learners will need longer than 24 weeks. Apply earlier when an eligible role matches work you can already demonstrate, rather than waiting for every row to be complete.

For an AI-product emphasis, swap the two flagship blocks after week 8 if your target roles justify it. For an MLE emphasis, keep forecasting first and deepen probability, optimization, SQL, and data pipelines. For research roles, treat mathematical depth and reproduction work as a later path, often with additional eligibility requirements.

The related learning roadmap can supply the learning sequence: guided foundations and Python/Git/data modules first; evaluation and classical ML for Demand Replay; retrieval/evaluation modules for Evidence Desk; packaging, serving, monitoring, and responsible-system modules for the reliability release. You do not need to finish every optional vision or research module before beginning a relevant project.

A six-hour active-work pattern

BLOCK	MINUTES	PURPOSE
Retrieve and plan	30	Reconstruct yesterday's concept and choose one observable outcome
Learn the next prerequisite	90	Read, work a small example, and answer a question before running code
Build or investigate	150	Implement one bounded behavior or reproduce one failure
Evaluate and test	60	Run meaningful checks and inspect actual outputs
Explain and record	30	Write the decision, result, limitation, and next step

The total is 360 active minutes; breaks are additional. Early weeks will need more guided learning; later weeks can transfer that time to implementation. On one day, use part of the documentation block for a real portfolio update. Do not turn every study session into a promotional post.

---

12. Readiness: what you should be able to do without a script

Use these as practice gates, not an external certification. Keep assistance and independent attempts distinct.

1. **Build a small change:** add a new validation rule, feature transform, or API behavior from a written requirement, with a meaningful test.
2. **Debug an unfamiliar failure:** trace a malformed record or retrieval miss through the system without guessing at random fixes.
3. **Explain the baseline:** describe why it is fair, where it fails, and whether complexity was justified.
4. **Defend the split:** identify time/entity/topic leakage and tell a reviewer what the final test does and does not establish.
5. **Read the metrics:** show numerator, denominator, sample size, slices, and uncertainty; identify a metric that would reward the wrong behavior.
6. **Trace a request:** explain the components, failure boundaries, logged metadata, model/data versions, and recovery path.
7. **Discuss tradeoffs:** name an alternative you rejected and the evidence that could change your mind.

8. **Explain your contribution:** distinguish original implementation, borrowed material, collaborators, and AI-assisted work.

A useful AI-assistance note is: “I used [tool] for [specific tasks]. I independently reviewed/implemented [components], ran [checks], and can explain [decisions].” Match it to the truth. Generated code is useful assistance; independent ability must be demonstrated separately.

Anthropic explicitly invites candidates to highlight independent research, thoughtful writing, and open-source work, and describes live coding. OpenAI's interview guide emphasizes reasoning, code quality, performance, test coverage, and communication. These are employer-specific examples, not promises of access to those companies. Their tool rules differ by assessment; follow the explicit instructions and ask the recruiter if unclear. S22 S23 S24

---

13. Turn the portfolio into an application process

Start a role inventory while learning. Once the first flagship is defensible, begin with a manageable set of genuinely relevant opportunities. For each role, record the employer URL/date, actual responsibilities, required versus preferred qualifications, eligible countries, authorization/sponsorship, language, time zone, degree/enrollment, and your strongest matching evidence.

Use three decisions:

- **Eligible and evidence-aligned:** tailor the application to two or three relevant behaviors and link directly to the proof.
- **Eligibility unclear:** investigate the employer's stated policy or ask a concise appropriate question; do not infer worldwide eligibility from “remote.”
- **Hard eligibility mismatch or substantially different seniority:** choose another opportunity or a longer-term preparation path. A portfolio cannot grant a required enrollment status, clearance, or work authorization.

Do not mechanically reject yourself for every preferred tool you lack. Do not ignore explicit hard conditions. A job-description wish list and a legal/student-status condition are different kinds of constraints. Keep the application truthful.

Resume bullet pattern: “Built [system] for [problem]; compared [baseline] and [candidate] on [N, split], obtaining [actual metric] under [measured constraint]. Added [specific failure handling/tests], with [main scope limit].” For a contribution: “Contributed [change] to [project]; [submitted/reviewed/merged], addressing [specific behavior and review feedback].”

Use a simple application log. If you get few screens, inspect eligibility, role alignment, and whether the evidence is visible. If you reach technical interviews but struggle, practice the observed gaps. If people cannot run your project, repair reproduction. Small samples are noisy; do not invent a causal explanation for each rejection or a conversion-rate guarantee.

The global goal is a direction, not a location filter called “anywhere.” A reader’s country and work authorization must be checked separately; this report does not establish eligibility for any sampled vacancy. Build capability, then test the opportunity set available to your actual situation.

---

14. What to postpone, and why

TEMPTATION	BETTER DEFAULT AT YOUR STAGE	WHEN TO RECONSIDER
A generic chat UI with no evaluation	A narrow assistant with a baseline, citations, abstention, and an error report	Broader scope after a real use case and evaluation support it
Training an LLM from scratch	Understand small models, then use an existing model behind a bounded interface	A research/infra goal with data, compute, math, and a meaningful experiment
Fine-tuning before defining success	Establish a baseline, failure taxonomy, clean data, and held-out evaluation first	A measured failure that adaptation could plausibly address
Multiple autonomous agents	A fixed, inspectable workflow with bounded tools	A task requiring dynamic decisions and an evaluation that justifies the extra failure surface
Kubernetes, feature stores, distributed training everywhere	One small service/container, explicit artifacts, simple CI and logs	A concrete role/project requirement or a demonstrated scaling problem
Dozens of cloned notebooks	Two independently understood systems with reproducible evidence	Small exercises remain useful for learning, without overstating their portfolio significance
A huge model score without context	Baseline, split, sample size, constraints, and failures	More complexity only when the decision requires it
Posting daily to appear active	Publish when a real experiment, fix, review, or release teaches something	Cadence can change with meaningful work and audience feedback

A later specialization should follow repeated requirements in your actual eligible role list. Computer vision, recommendations, speech, multilingual NLP, and research replication can all be valuable. Choose one when you have a domain question, appropriate data rights, a measurable task, and a reason that it adds evidence your existing work lacks.

---

15. Your next seven days

Begin with ownership of one small program before starting the flagships.

DAY	CONCRETE WORK	EVIDENCE TO KEEP
1	Practice running Python, save a small program, and explain each line	Saved file, actual output, and a short explanation in your own words
2	Practice functions, loops, and dictionaries on a tiny invented dataset	One small change made without copying, plus its expected/actual output
3	Read a ten-row CSV fixture and summarize its fields/counts	The fixture, parser, output, and one malformed-input example
4	Add date/count checks and clear errors	Tests for good input, missing columns, invalid counts, and duplicates
5	Use Git meaningfully; write run/setup instructions and a scope statement	A clean-clone check and a README another person can follow
6	Add one new requirement independently; inspect three real role descriptions	A new regression test and an eligibility/evidence worksheet
7	Rest or do light review; identify what is still not independently understood	A short next-week plan based on actual gaps, not a completion streak

If those prerequisites take longer, use more days. Do not claim that a seven-day exercise makes you ready for an AI-engineering job. Its purpose is to establish the working method that makes the later projects credible.

The immediate objective is precise: **finish one small program you can run, test, repair, and explain.** The longer-term objective is a small body of work where each claim has a visible result, a valid evaluation, and a contribution you genuinely understand.

EVIDENCE APPENDIX · HIRING

What the postings actually ask for.

Inspect the role, geography, seniority, and qualification boundaries alongside the skills. A posting is a dated observation; a portfolio does not replace its eligibility requirements.

The 20 postings are a purposive sample from 11 employers, including 17 AI/ML roles and three adjacent software roles. They are evidence of described work and eligibility, not a representative global survey. One source contains unfinished templates. Original publication dates were not inferred from feed update timestamps.

H01 · Databricks — AI Forward Deployed Engineer

London, United Kingdom · Customer-facing applied AI / forward deployment

Advert says all levels, but explicitly asks for minimum four years.

Explicit requirements / eligibility

- Minimum four years in data science, ML, or AI.
- Graduate quantitative degree or equivalent practical experience.
- Willingness to travel to customers every four to eight weeks as needed.

Other listed qualifications

- Production GenAI integration, evaluation and optimization.
- Cloud ML deployment, technical communication, and collaboration.

Preferred / nice to have

- Databricks platform and Spark experience explicitly marked preferred.

Location boundary: London role. Worldwide remote access and sponsorship are not established by this description.

> “Min 4 years experience in DS, ML, AI”

> “Expertise in deploying production-grade GenAI applications, including evaluation and optimizations”

Portfolio implication: Show one customer problem, a deployed AI workflow, evaluation cases, latency/cost trade-offs, and a clear technical explanation. This does not substitute for four years of requested experience.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.856542+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H02 · Databricks — Applied AI Engineer

Belgrade, Serbia · Applied ML/AI product and infrastructure

Experienced engineer; explicit range of two to eight years.

Explicit requirements / eligibility

- Two to eight years of engineering experience is listed.

Other listed qualifications

- Strong software engineering, testing, and deployment.
- Production ML infrastructure, training/serving components, and end-to-end systems.

Preferred / nice to have

- Production modeling beyond standard libraries described as a plus.
- Deployment, scaling, monitoring, and critical-service infrastructure experience described as nice to have.

Location boundary: Belgrade role. No degree, sponsorship, or worldwide-remote eligibility rule is established in the selected description.

> “Infrastructure and system engineers with no ML background and willingness to excel in personal growth are welcome to apply.”

> “Strong software engineering skills, and familiarity with software engineering principles around testing and deployment.”

Portfolio implication: Software engineering can be a bridge into applied AI. Build reproducible tests, an API, data contracts and operational behavior; do not misread no ML background as no engineering experience.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.856542+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H03 · Datadog — AI Research Engineer - Datadog AI Research (DAIR)

Paris, France · Research engineering / ML systems

Advanced practical systems and research experience; numeric minimum not specified.

Other listed qualifications

- Python plus familiarity with a systems language.
- Distributed training/inference, GPU acceleration, practical large-model training/fine-tuning and research contribution experience.
- Clear explanation of performance and design trade-offs.

Preferred / nice to have

- CUDA, observability/SRE/security, production data pipelines and simulation environments are among bonus items.

Location boundary: Paris role. No numeric experience minimum, specific degree requirement, or sponsorship guarantee is stated in the selected text.

> “Establish rigorous automated benchmarks and regression tests for world model predictions, agent performance, and simulation fidelity”

> “You can explain design and performance trade-offs clearly to both technical and non-technical audiences”

Portfolio implication: A later research-oriented artifact could reproduce an experiment and profile a training/inference bottleneck. Large-scale distributed RL is a specialization, not a sensible first project.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.857284+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H04 · Elastic — Agentic AI Engineer

United States · Enterprise agentic AI engineering

Advanced breadth is requested; no junior title or numeric experience minimum.

Other listed qualifications

- Advanced Python or TypeScript; practical RAG/hybrid search, tools, structured outputs, and agent state.
- Cloud infrastructure/IaC, CI/CD, security/identity, tracing, token costs and output evaluation.
- Proven multi-agent implementation experience.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: United States posting. Distributed-company language is not evidence that this role is open from every country. Export-control language may affect employment; sponsorship is not stated.

> “They will also help manage token spending and identify model drift or agent loops.”

> “Advanced proficiency in Python or TypeScript for backend orchestration, agent memory systems, and API service development.”

Portfolio implication: Demonstrate bounded tool execution, state transitions, schema validation, traces, cost limits and safe failure. Establish a single-workflow baseline before adding multi-agent complexity.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.857693+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H05 · Anthropic — Applied AI Demo Engineer

San Francisco, CA | New York City, NY · Applied AI demo / product engineering

Experienced cross-functional role; exact years depend on internal level.

Explicit requirements / eligibility

- Bachelor’s degree or an equivalent combination of education, training, and experience.
- Relevant field can be shown through coursework, training, or professional experience.
- Office presence at least 25 percent under stated general hybrid policy; some roles may require more.

Other listed qualifications

- End-to-end Python/TypeScript software; LLM APIs, tools/retrieval and third-party integrations.
- Reliable prototypes, reusable assets, evaluation harnesses, realistic sanitized data and technical storytelling.

Preferred / nice to have

- The advert uses a broad good-fit list rather than a separate required/preferred partition; do not convert every item into a hard minimum.

Location boundary: San Francisco or New York. The employer says it sponsors visas, but explicitly cannot sponsor every role/candidate. This is not an individual sponsorship promise.

> “accuracy and honesty matter as much as polish”

> “Experience building evaluation or validation harnesses for AI systems, including reliability checks and graceful fallbacks”

Portfolio implication: A polished demo should disclose what works and fails, use safe data, reproduce setup, and demonstrate graceful fallbacks. This is direct evidence against presenting visual polish alone as AI expertise.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.858092+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H06 · Reddit — Machine Learning Engineer

Remote - United States · Production ML engineering across consumer and ads

Experienced role; basic qualifications list three to five-plus years.

Explicit requirements / eligibility

- Three to five-plus years building, deploying, and operating production ML systems.

Other listed qualifications

- Software fundamentals, classic and deep-learning foundations, scalable pipelines and serving.
- Full lifecycle ownership, measurable metrics, reliability, observability, retraining and cross-functional problem definition.

Preferred / nice to have

- Recommenders/search/ads/representations, distributed tools, low latency, LLM methods and advanced degree are preferred.

Location boundary: Remote – United States. The word remote does not establish global eligibility; no sponsorship conclusion is drawn.

> “3-5+ years of experience building, deploying, and operating machine learning systems in production”

> “Own the full ML lifecycle: from problem definition and feature engineering to training, evaluation, deployment, and monitoring”

Portfolio implication: A classical ML project should include data boundaries, baseline, held-out evaluation, serving and monitoring rather than stopping at notebook accuracy. A personal replay is not production employment.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.858378+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H07 · Pinterest — Machine Learning Engineer, Core Engineering

San Francisco, CA, US; Palo Alto, CA, US; Seattle, WA, US; Remote, US · Applied recommendations / retrieval ML engineering

Two-plus years industry ML experience is listed.

Explicit requirements / eligibility

- Two-plus years industry experience applying ML methods.

- Bachelor’s degree in a related field or equivalent experience.

Other listed qualifications

- End-to-end data processing pipelines and large-scale ML systems.
- Experimentation and candidate retrieval for product surfaces.

Preferred / nice to have

- Master’s/PhD, publications, AI coding assistants, LLM productivity tools and scalable realtime systems are nice to have.

Location boundary: US-based applicants only; no relocation assistance. Location metadata includes named US offices and Remote, US.

> “2+ years of industry experience applying machine learning methods”

> “US based applicants only”

Portfolio implication: If targeting search/recommendations, publish a retrieval baseline, ranking evaluation, error slices and reproducible pipeline. Explain and verify AI-assisted changes yourself.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:44.319856+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H08 · Pinterest — Machine Learning Intern 2027 (Dublin)

Dublin, IE · ML internship

Student internship with graduate enrollment requirement.

Explicit requirements / eligibility

- Working toward a master’s or PhD in a related field.
- Commutable to Dublin for two to three office days a week.
- Full-time internship dates May 10–August 27, 2027.

Other listed qualifications

- Applied ML experience may come from research, coursework, projects or internships.
- Programming/framework proficiency, research/analytical work, maintainable documented/tested code and communication.

Preferred / nice to have

- No separately labeled preferred qualifications section. Relocation assistance may be available; sponsorship is not stated.

Location boundary: Dublin hybrid internship. Graduate enrollment, location and dates must be checked separately from portfolio quality.

> “Working towards a Master's or PhD degree in Computer Science, ML, NLP, Statistics, Information Sciences or related field”

> “Experience applying machine learning or AI concepts through research, coursework, projects or internships.”

Portfolio implication: A scoped project covering problem definition through implementation and results presentation matches intern work. Projects can evidence skills here, but cannot waive enrollment requirements.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:44.319856+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H09 · Pinterest — Master's University Grad Machine Learning Engineer 2027 (USA)

San Francisco, CA, US; Palo Alto, CA, US; Seattle, WA, US; New York, NY, US · Graduate ML engineering

Master’s university graduate cohort, title specifies 2027.

Explicit requirements / eligibility

- A relevant master’s degree is explicitly required.
- US-based applicants only.

Other listed qualifications

- ML methods, big data/realtime systems, a systems language or ML framework.
- Research/analytical ability and validation of AI-generated code for correctness, performance, security and maintainability.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: Named US offices. The role may require commuting distance for in-person collaboration. Do not infer a precise graduation window absent from the body.

> “Master's in Computer Science, ML, NLP, Statistics, Information Sciences or related field required”

> “Strong critical thinking when working with AI-generated suggestions, with a clear approach to validating correctness, performance, security, and maintainability.”

Portfolio implication: Show a reproducible model study and the tests used to accept or reject AI-generated suggestions. This posting is not evidence that a non-graduate beginner can bypass its master’s requirement.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:44.319856+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H10 · Roblox — [2027] Senior Machine Learning Engineer - PhD Early Career

San Mateo, CA, United States · Research-oriented ML engineering

Title combines senior ML engineer with PhD early career.

Explicit requirements / eligibility

- Possess or be pursuing a relevant PhD with aligned thesis.

Other listed qualifications

- Deep relevant domain expertise, programming and large-scale optimization.
- Multiple peer-reviewed publications/presentations are listed as evidence of a strong research track record.

- End-to-end research-to-production work.

Preferred / nice to have

- No separately labeled preferred section.

Location boundary: San Mateo office. Office-based roles attend Tuesday–Thursday. The posting cautions that certain US work-authorization categories or future H-1B sponsorship may not be supported.

> “Possess or pursuing a PhD in computer science, engineering, or a related field, with a thesis aligned to Roblox’s research areas.”

> “A strong research track record, evidenced by multiple publications and presentations in rigorous peer-reviewed venues.”

Portfolio implication: Research replication is a later, role-specific route. The early-career label here describes PhD career stage, not suitability for someone starting programming.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:44.492851+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H11 · Duolingo — AI Research Engineer, New PhD Graduate

New York, NY; Pittsburgh, PA · AI research engineering

New PhD graduate role.

Explicit requirements / eligibility

- Relevant PhD completed or expected by August 2027.
- Ability to relocate to Pittsburgh or New York.

Other listed qualifications

- Impactful portfolio through publications, open-source projects or AI internships.
- Debugging, critical thinking, interdisciplinary R&D and communicating findings.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: New York or Pittsburgh. A relocation ability is explicitly requested; sponsorship is not stated in the captured description.

> “You have an Ph.D. in computer science or related technical field (or expect to by August 2027)”

> “A portfolio of impactful work, in the form of (for example) publications, open-source projects, or AI internships”

Portfolio implication: This explicitly supports meaningful public work as evidence, while also showing that portfolio and degree requirements can coexist.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:44.690845+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H12 · Airbnb — Machine Learning Engineer, Assistance Engineering

China · Customer-support ML / GenAI engineering

Two to four years ML engineering experience is listed.

Explicit requirements / eligibility

- PhD/master’s degree or equivalent experience.
- Two to four years ML engineering experience with large-scale software ownership.
- English and Mandarin fluency is essential.

Other listed qualifications

- Scalable AI/ML systems, product integrations, performance and cross-functional collaboration.

Preferred / nice to have

- LLM chatbot and agentic product experience is described as a big plus.

Location boundary: China-based; permitted city depends on legal working identity and company policy. The description does not establish global remote eligibility.

> “2-4 years of ML engineering experience, with ownership responsibility over large-scale software systems.”

> “Fluency in both English and Mandarin is essential.”

Portfolio implication: An evidence-grounded support workflow with evaluation and guardrails is relevant. Language and legal-location requirements remain separate filters.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.064925+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H13 · Scale AI — Applied AI Engineer

Kuwait City, Kuwait · Applied AI component / backend integration

Scoped engineering ownership; no numeric experience minimum specified.

Other listed qualifications

- Production software experience; Python and another language.
- Model/API/tool integration, ML evaluation and regression diagnosis.
- Own implementation through testing, release, production debugging and client communication.

Preferred / nice to have

- Backend/full-stack background, cloud ML, agentic/RAG/context engineering, regulated data experience.

Location boundary: Kuwait City. No explicit degree minimum or sponsorship/remote policy is stated in this description. Absence of a requirement is not proof it is waived.

> “Run and extend evaluations within an established framework, interpret results, investigate common failure modes, and flag regressions”

> “Experience owning a defined piece of work through testing, release, and production debugging”

Portfolio implication: Choose a tractable AI component and show its API, tests, eval set, failure investigation and release history. This is a closer portfolio-shaped scope than attempting a frontier model, but production experience is still requested.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.077324+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H14 · Scale AI — Machine Learning Engineer, Public Sector

Denver, CO; Honolulu, HI; Washington, DC · Public-sector production ML engineering

Experienced production ML profile; experience list is under ideally-have heading.

Explicit requirements / eligibility

- Active TS security clearance is explicitly required.
- Comfort with about ten percent travel.

Other listed qualifications

- Production models, retraining, scalable infrastructure and product/research collaboration.

Preferred / nice to have

- Two-plus years production applied ML and strong algorithms/programming appear under Ideally You’d Have.
- Fine-tuning, evaluations, constrained deployment, government experience and post-launch ownership appear under Nice to Haves.

Location boundary: Denver/Honolulu/Washington DC locations. Active clearance is a hard stated access barrier; do not treat a public portfolio as a substitute.

> “This role will require an active TS security clearance”

> “A shipped project with real users behind it, where you owned it after launch rather than handing it off at merge.”

Portfolio implication: The general evidence lesson is post-launch ownership, regression checks and operational maintenance. This is not an eligible target for someone without the required clearance; no defense-specific project is recommended.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.077324+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H15 · Scale AI — ML Research Engineer, ML Systems

San Francisco, CA; Seattle, WA; New York, NY · ML systems / research infrastructure

Specialized distributed ML systems; qualifications framed as ideally-have.

Other listed qualifications

- Build, profile and optimize training/inference infrastructure.
- Collaboration across ML/research teams.

Preferred / nice to have

- Multi-node LLM training/inference, distributed ML systems, CUDA/PyTorch/transformers and communication listed as ideal.
- Post-training methods and next-generation LLM use cases are nice to have.

Location boundary: San Francisco, Seattle or New York. Degree, numeric minimum and sponsorship are not specified in the captured text.

> “Build, profile and optimize our training and inference framework”

> “Experience with multi-node LLM training and inference”

Portfolio implication: A specialist track would require actual profiling, throughput/memory measurements and a reproducible optimization experiment. Do not start by buying multi-GPU infrastructure.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.077324+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H16 · Affirm — Machine Learning Engineer II (Underwriting ML)

Remote Canada · Tabular/sequential underwriting ML engineering

ML Engineer II; two-plus years ML experience or relevant PhD.

Explicit requirements / eligibility

- Two-plus years as an MLE or a relevant PhD.
- Bachelor’s degree in a related field or equivalent practical experience.
- Residence in one of the expressly listed Canadian provinces.

Other listed qualifications

- Production Python, classification, pipelines, offline experiments, model serving, monitoring, backtesting and code review.
- AI-powered developer tools with code-quality judgment.

Preferred / nice to have

- Gradient-boosted trees preferred for classification; PyTorch and Spark explicitly preferred among framework options.

Location boundary: Remote Canada only: Alberta, British Columbia, Manitoba, New Brunswick, Newfoundland and Labrador, Nova Scotia, Ontario, Prince Edward Island, or Saskatchewan. Not worldwide remote.

> “You have a total of 2+ years of experience as a machine learning engineer or a PhD in a relevant field.”

> “You will instrument and monitor model and data health, and help define retraining/backtesting workflows”

Portfolio implication: A careful tabular decision project with validation boundaries, cost-aware evaluation, feature pipeline, serving and monitoring maps directly to this work. A historical demo must not claim real credit-risk readiness.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.079090+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H17 · Affirm — Software Engineer (Machine Learning) Intern (Summer 2027)

San Francisco, California, United States · ML-labeled software engineering internship

Summer 2027 student internship; degree level is not precisely specified.

> **Source warning:** The source contains unfinished compensation, geography, remote-work and sponsorship templates. A no-sponsorship sentence appears both before the template block and inside a conditional instruction. This record is excluded from sponsorship/remote eligibility conclusions in the main analysis; verify the current official listing. Placeholder pay ranges are not usable evidence.

Explicit requirements / eligibility

- An unqualified no-sponsorship sentence appears before the unfinished template block; because the same captured document retains conditional sponsorship instructions and other placeholders, confirm the applicable policy before using this role as an eligibility example.

Other listed qualifications

- Python/C++/Java, frontend experience, OOP and communication.
- Testing/deployment frameworks, API documentation and cloud/PaaS.
- Ship a project, verify it in production, monitor it and present results.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: Listed as San Francisco, summer 2027, twelve-to-sixteen-week internship. The text targets students without a precise degree/enrollment rule. Template contamination makes some conditions unreliable; do not infer worldwide access or a definitive sponsorship policy from the conditional boilerplate.

> "Interns are responsible for developing and shipping a project and verifying that it works in production."

Portfolio implication: Internship-style proof is a small component delivered and maintained with tests, deployment and a presentation. Do not infer global access from an international company name.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.079090+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H18 · Affirm — Software Engineer I, Early Career (London)

London · Adjacent early-career software engineering

Early Career / Software Engineer I.

Explicit requirements / eligibility

- Bachelor’s degree in a related field or equivalent practical experience.
- Visa sponsorship unavailable.
- Hybrid London office attendance three days per week during the early-career program.

Other listed qualifications

- Turn a problem into multiple interacting software components with clear, tested, extensible code.
- Navigate/debug an existing codebase, review code and communicate trade-offs.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: London hybrid for the first year. Later remote flexibility remains within the country-of-employment framework; no worldwide remote eligibility is implied.

> “This position requires either equivalent practical experience or a Bachelor’s degree in a related field.”

> “Please note that visa sponsorship is not available for this position.”

Portfolio implication: An adjacent software role can value the same API, testing, code review and debugging evidence needed for AI products. Equivalent practical experience is explicitly allowed, but work authorization remains a separate gate.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:45.079090+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H19 · Databricks — Software Engineer - New Grad (2027 Start) - London

London, United Kingdom · Adjacent new-graduate software engineering

New-grad 2027 cohort.

Explicit requirements / eligibility

- Graduate between autumn 2026 and summer 2027 with a related degree.

Other listed qualifications

- General-purpose programming, algorithms/data structures/OOP and end-to-end projects.
- Design, test and operate reliable, scalable and secure software.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: London role. Graduation window is explicit; sponsorship is not established. Employer asks candidates to select one preferred office rather than submit multiple location applications.

> “You will graduate between autumn 2026 and summer 2027 with a degree in Computer Science, Engineering, or related subject”

> “You have experience managing end-to-end projects”

Portfolio implication: A well-scoped end-to-end application with readable design, tests and operational evidence is useful preparation for software entry routes when the candidate meets the graduation window.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.856542+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

H20 · Datadog — Software Engineering Intern

Madrid, Spain · Adjacent software engineering internship

Student internship with country/university and timing constraints.

Explicit requirements / eligibility

- Expected 2027 graduation in a relevant degree from a university in Spain.
- Availability for a February 2027 internship start; no other start dates supported in the text.

Other listed qualifications

- CS fundamentals, programming and productive use or interest in AI-assisted development.
- Deliver technical projects through deployment with guidance; reliability/performance collaboration.

Preferred / nice to have

- No separately labeled preferred qualifications section.

Location boundary: Madrid hybrid role, available full-time or part-time under stated program. Relocation support may apply, but the university-location requirement is explicit; sponsorship is not established.

> “with a degree in Computer Science, Software Engineering, or a related technical field from a university in Spain”

> “Demonstrate strong computer science fundamentals, including data structures, algorithms, and software design”

Portfolio implication: An early software project should also show basic algorithms, data modeling and design choices, not only API prompting. Student and program-location constraints may make this role ineligible.

Original employer listing · Employer-controlled feed

Captured 2026-11-06T17:34:43.857284+00:00; reviewed 2026-11-07. Listed in the feed at capture time. This does not guarantee the role remains open or that this candidate is eligible. Feed update time is not original publication time.

EVIDENCE APPENDIX · SOURCES

Follow the reasoning back to its source.

Source-backed observations, planning estimates, and recommendations serve different purposes. Keep the source dates and sampling limits attached to the claims.

These 25 source pages support the observations and technical guidance. The project choices, schedule, budgets and readiness gates are authored recommendations. Duplicate captures were not counted as extra sources. A LinkedIn FAQ returning 404, a Google hiring page containing only boilerplate, and unavailable Creative Commons legal pages were not used as evidence. The UCI page itself states the dataset license. Raw captures are retained in the working research directory; the toolkit includes analytical summaries, short attributed excerpts and hash ledgers instead of full third-party pages.

S01. US BLS — Data scientists. Accessed 2026-11-07. US occupational projection; not global junior AI vacancies.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:48.443952+00:00.

SHA-256: `140c4c267b1648b16b458503c163ded1a9cf11a36e395d538557c09c8f0246a7`

S02. US BLS — Software developers, QA analysts, and testers. Accessed 2026-11-07. Broad combined US occupational group; not an AI-specific forecast.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:48.444902+00:00.

SHA-256: `adfa37ddf3d56f9c37d80ed4e60cd6ad49e9fcf07c6a21509368339d46e78ada`

S03. World Economic Forum — Future of Jobs Report 2025. Accessed 2026-11-07. Employer expectations through 2030; not a probability of getting hired.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:48.445274+00:00.

SHA-256: `9ddf084eb058d02af09528fba43551c37e86c89bad677b405df73acd26f4fd91`

S04. Stanford AI Index 2026 — Economy. Accessed 2026-11-07. Broad economic and labor evidence; uneven effects and limits to causal attribution.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:48.446059+00:00.

SHA-256: `d5694021b95586cfba0695c7c56687828a52771e2455d5d4ecec98ccc7efeea2`

S05. Google — Rules of Machine Learning. Accessed 2026-11-06. Engineering guidance on baselines, metrics and dependable pipelines.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `93c4c4935a9b0f40385d2aa9712bde9f1deab53b4dccbfb2f06330ea239f11ee`

S06. Scikit-learn — Common pitfalls and recommended practices. Accessed 2026-11-06. Preprocessing and leakage guidance; the prediction-time contract still needs explicit design.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `21fac11c181ab35624dcf8673f0475d403d4f6ecf0f9bead01a573d7642b515c`

S07. Scikit-learn — TimeSeriesSplit. Accessed 2026-11-06. Chronological cross-validation mechanics; does not itself prevent feature leakage.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `bd5b51d06ac0ab50b62badb7d35de9bb2bdaa10a059235b3bf19cd17b57cdcc3`

S08. OpenAI — Evaluation best practices. Accessed 2026-11-06. Task-specific evaluation guidance; model judges require validation.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `0c0106b575b6886fb11cdea6a5c01b7f22a5ea19d23fbd147fd5f792196a3944`

S09. Anthropic — Building effective agents. Accessed 2026-11-06. Distinguishes fixed workflows and agents; supports starting with simple composable systems.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `c3840081c5d8701cdea29cf72b98fec648aabf29fbdbac866107ba08518fe975`

S10. OWASP — LLM01: Prompt injection. Accessed 2026-11-06. Threat and mitigation guidance; a small attack set is not a security certification.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `8e36a5dff842ed950a70a3b1a05ea47f4ccd6bee82b92a9d488b711c3b511617`

S11. Google Research — The ML Test Score. Accessed 2026-11-06. Publication overview on ML production readiness and technical debt.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `5314f45d047bd21c483df1493bc12f3fe8aa77572fdf6618688f660880e7905f`

S12. Evidently — Data drift. Accessed 2026-11-06. Distribution-shift monitoring; an alert alone does not establish predictive degradation.

Capture provenance

HTTP 200; retrieved 2026-11-06.

SHA-256: `8945a14408ee6b16c3b22f394841e0b131fe5002327c2d35a774a4b5bfdf7c80`

S13. UCI — Bike Sharing Dataset. Accessed 2026-11-07. Official dataset description and CC BY 4.0 license statement; actual ZIP was separately inspected.

Capture provenance

HTTP 200; retrieved 2026-11-07.

SHA-256: `7462eb10852ea268a3f5db0f2fb5f21940b1570a22083ec90de8ad706877733d`

S14. Python — History and License. Accessed 2026-11-07. PSF license for documentation; specified examples additionally available under 0BSD. Preserve applicable notices.

Capture provenance

HTTP 200; retrieved 2026-11-07.

SHA-256: `903ff8c671a2116fbb07ab041aa9322dc5882353fa5d31b45e33ddf7f8fe57c6`

S15. Python — Download documentation. Accessed 2026-11-07. Official downloadable documentation. Pin exact version/artifact/hash; /3/ is a moving URL.

Capture provenance

HTTP 200; retrieved 2026-11-07.

SHA-256: `4775df7a79bb5fbac3389a14152786829c9039f46d05921e42ace083cecdc2e6`

S16. GitHub — About READMEs. Accessed 2026-11-07. README content and presentation guidance, not a hiring guarantee.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:13.910410+00:00.

SHA-256: `e972115a096e6c339402c864cebbd7b36d999a7675dc9fa25f4e655b407fa932`

S17. GitHub — Pinning items to your profile. Accessed 2026-11-07. Up to six repositories and gists combined can be pinned.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:13.909998+00:00.

SHA-256: `90d94ffcd73a47ad59b916dce4797d36d8183ea489648bb9be5d0260eb481c68`

S18. GitHub — Managing your profile README. Accessed 2026-11-07. Profile README repository conventions and eligibility.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:13.909315+00:00.

SHA-256: `860caa390a30421d62ab02899a89e39efa5e1b21ec60e5f4dc3e47f52c83ad78`

S19. GitHub — About continuous integration. Accessed 2026-11-07. Automated checks and results; passing CI only covers the configured checks.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:13.910879+00:00.

SHA-256: `7eae25a16329cd6f989ac75e8e42a196d099f9e155697f06b8a1cfeb79cfd46d`

S20. GitHub — Removing sensitive data from a repository. Accessed 2026-11-07. Revoke/rotate exposed credentials; removal does not erase uncontrolled copies.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:14.078413+00:00.

SHA-256: `c1e8d8dbba2f3a2720a03ff06e78bef777ef8f9cd4d7d870bb2c58cd89366858`

S21. LinkedIn — Manage Featured samples of your work. Accessed 2026-11-07. Featured supports posts, articles, links and media. No ranking or posting-time claim.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:14.341549+00:00.

SHA-256: `a12b4c90c085c63bd4707ec25399c85b4cc7991af780a20fafaad2cd2a4ffe94`

S22. Anthropic — Careers. Accessed 2026-11-07. One employer values independent research, technical writing and open source; not a universal entry-level rule.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:48.863112+00:00.

SHA-256: `426afcec227a7a2145dfc04a882cd66962d73de85999788973ba38cb8ae185eb`

S23. Anthropic — Candidate AI guidance. Accessed 2026-11-07. Employer-specific AI-use guidance. Follow actual assessment instructions.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:14.607700+00:00.

SHA-256: `7803fa8eb5c0f5fae8a6ad3c27cbd05a1cb91d98c099dba9ddf0b904a98e4ee9`

S24. OpenAI — Interview guide. Accessed 2026-11-07. Employer-specific skill assessment and communication guidance; tool rules vary by format.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:48.685804+00:00.

SHA-256: `887f58285bf0d3cf9a59df7cff304a7d8439de68d17cdbed85d6614c8a4fe9fb`

S25. GitHub Open Source Guides — How to contribute. Accessed 2026-11-07. Contribution etiquette, project conventions and useful scope.

Capture provenance

HTTP 200; retrieved 2026-11-07T06:40:14.286849+00:00.

SHA-256: `8ff70a4586ba12ce3fd4c02ce0250041773086b9b809728aa60b622c19d2c036`

Dataset feasibility check

Official dataset archive inspected 2026-11-07T12:05:49.042976+00:00: 731 daily rows and 17,379 hourly rows. Archive SHA-256: b70182d0d0508e9abbb79306ce5c0cec34869000f8220175ac83d11dbe845401. This inspection established schema feasibility only; no model was trained.

Official UCI archive

One useful project. Evidence you can explain.

This is a plan for building and presenting credible work. Project scopes and study budgets are recommendations; hiring outcomes depend on demonstrated ability, eligibility, fit, and available roles.